

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

ReservaLab

Fase 2 · Automatización y plataforma

E9-E16: pruebas unitarias y API, cobertura, SonarQube, pipeline, carga y defensa.

ERICK ESTUARDO CRUZ ROMERO

Carné: 0900-19-1362

Catedrático: Ing. Michael Rodolfo Asturias López

Curso: Aseguramiento de la Calidad de Software

Segundo semestre de 2026

Pruebas locales ejecutadas; integración externa y carga cloud pendientes.

Preparación: 25 de septiembre de 2026. El estado de cada evidencia se declara dentro del documento.

Índice

1. E9 · Estrategia de ramas	3
1.1 Reglas de protección pendientes de configurar	3
1.2 Tres cambios propuestos para revisión real	3
2. E10 · Pruebas unitarias	4
2.1 Línea base e incremento	4
2.2 Dobles de prueba	5
3. E11 · Pruebas de API	6
4. E12 · SonarQube	7
4.1 Ejecutar en Windows	7
4.2 Hallazgos de dependencias: medición distinta	7
5. E13 · Pipeline	9
5.1 Cloud con persistencia	9
6. E14 · Carga	10
6.1 Conclusión que debe completarse con la medición	10
7. E15-E16 · Defensa y bitácora	11
8. Estado por entregable	11
9. Fuentes técnicas	12

1. E9 · Estrategia de ramas

Rama	Propósito	Promoción
DEV	Integración de cambios y pruebas iniciales	feature/* -> DEV mediante PR.
QA	Candidato para validar en cloud y ejecutar carga	DEV -> QA después de pipeline aprobado.
main / PROD	Versión estable de la demostración	QA -> main después de validación, revisión y evidencias.

Los ambientes deben tener bases y servicios separados. El workflow selecciona el ambiente por rama destino y conserva el SHA exacto al desplegar. Los secretos no se guardan en Git. Los Pull Requests ejecutan calidad pero no despliegan; un push a una rama autorizada despliega después de todas las comprobaciones.

1.1 Reglas de protección pendientes de configurar

- Proteger QA y main: exigir PR, check quality aprobado y rama actualizada.
- Exigir revisión efectiva por otra persona autorizada; impedir autoaprobaciones y push directo.
- Resolver todos los comentarios antes del merge y no eludir las reglas con permisos administrativos.
- Conservar al menos tres PR con revisión sustantiva y evidencia de que un pipeline fallido bloquea el merge.

El estudiante trabaja solo. La sección E9 exige otro integrante como revisor; se debe pedir una adaptación al catedrático antes de afirmar cumplimiento. El historial entregado no contiene revisiones ficticias ni tres PR creados artificialmente.

1.2 Tres cambios propuestos para revisión real

PR propuesto	Qué debería revisar otra persona
PR-01 · Validación de entradas	Revisar DEF-01, mensajes, HTTP 400/413, contratos y regresión CP-50/53/54.
PR-02 · Integridad de préstamos	Examinar bloqueo transaccional, casos concurrentes y cierre de DEF-02 con CP-49/51/52.
PR-03 · Automatización y despliegue	Revisar orden del pipeline, Quality Gate, secretos, aislamiento y despliegue del SHA verificado.

2. E10 · Pruebas unitarias

Vitest ejecutó 90 pruebas: domain.test.js (59), security.test.js (6), rules.test.js (22) y api.test.js (3).

Backend: 65; frontend: 25. Las pruebas unitarias verifican funciones y políticas aisladas; no se presentan las solicitudes de Newman ni la suite HTTP como pruebas unitarias.

Archivo del alcance	Líneas cubiertas	Alcance
backend/src/domain.js	100%	Unitaria
backend/src/security.js	100%	Unitaria
frontend/src/api.js	100%	Unitaria
frontend/src/rules.js	100%	Unitaria

El 100% corresponde al alcance explícito de cuatro archivos: domain.js, security.js, rules.js y api.js.

Quedan fuera de esa cifra app.js, db.js, server.js y main.jsx. Estos se verifican en pruebas funcionales/integración o navegador; no se afirma cobertura unitaria global.

2.1 Línea base e incremento

Módulo base	Antes	Después	Diferencia
backend/src/domain.js	0%	100%	+100 puntos
frontend/src/rules.js	0%	100%	+100 puntos

Línea base: cero pruebas de origen, preservada en baseline-v0. El reporte base usa c8 sin ejecutar pruebas y el reporte final usa Vitest/V8; ambos se comparan por porcentaje de líneas del mismo módulo, no por igualdad de contadores físicos. La mejora comprobada supera los 20 puntos exigidos en ese alcance. La ampliación del alcance a orquestación transaccional requiere pruebas unitarias adicionales y debe acordarse si así lo interpreta el catedrático.

```
npm run test:coverage
node scripts/check-coverage.mjs
```

2.2 Dobles de prueba

Prueba	Dependencia aislada	Justificación
MOCK-01	Repositorio: equipo disponible y sin conflicto	Verificar decisión de aprobación sin DB ni red.
MOCK-02	Repositorio: equipo en mantenimiento	Verificar rechazo y que no se consulta disponibilidad de agenda innecesariamente.
MOCK-03	Repositorio: conflicto existente	Verificar bloqueo por negocio sin fabricar filas en una base real.
MOCK-04	Repositorio que falla	Verificar propagación del error de infraestructura, sin confundirlo con una aprobación.
Cliente API	fetch simulado	Verificar envío de token/cuerpo y manejo de errores sin conectarse al servidor.

3. E11 · Pruebas de API

La colección versionada contiene 24 solicitudes y 85 aserciones automáticas de código HTTP, esquema JSON y contenido. Encadena registro, login de estudiante, reserva, login administrador, aprobación, entrega, devolución y auditoría. También cubre controles de permisos, duración, mantenimiento, duplicados y cierre de sesión.

Métrica	Resultado
Solicitudes	24 ejecutadas / 0 fallidas
Aserciones	85 ejecutadas / 0 fallidas
Framework	Postman Collection v2.1 ejecutada por Newman
Base / servidor	PGLite en memoria y HTTP local en puerto efímero
Reporte	qa/evidencias/api-newman.json
Limitación	No mide rendimiento cloud y no acredita ejecución del pipeline remoto.

```
npm run test:api
```

Cada ejecución crea un entorno aislado. La colección no debe ejecutarse contra PROD porque registra usuarios, cambia estados y crea datos. Los tokens no se escriben en el reporte resumido entregado.

4. E12 · SonarQube

El estudiante dispone de SonarQube en <http://localhost:9000>. No se pudo acceder a ese servidor desde la sesión de preparación: localhost identifica cada equipo. Se entregan la configuración, el script de escaneo y el exportador de métricas; no existen aún resultados iniciales/finales de Sonar ni un Quality Gate aprobado.

Dato exigido por E12	Estado
Bugs, vulnerabilidades, code smells	Pendiente de análisis Sonar real.
Duplicación y deuda técnica	Pendiente; registrar valores y unidad (deuda en minutos).
Blocker / Critical iniciales	Pendiente de consultar; no asignar cero por ausencia de reporte.
Remediación antes/después	Ejecutar, corregir hallazgos reales y comparar con evidencia.
Quality Gate	Configurado para esperar resultado; aprobación no verificada.

4.1 Ejecutar en Windows

Crear proyecto reservab-dev y generar token. Definir SONAR_TOKEN como variable de entorno solo en la sesión. Generar cobertura. Desde la raíz del proyecto, ejecutar scripts/sonar.ps1. El scanner se ejecuta en Docker y usa host.docker.internal:9000 para acceder a Sonar en el host Windows.

```
npm run test:coverage
./scripts/sonar.ps1 -ProjectKey reservab-dev
node scripts/sonar-evidence.mjs inicial
```

Configurar un Quality Gate con cero Blocker/Critical abiertos, cobertura $\geq 80\%$ dentro del alcance acordado y duplicación $\leq 3\%$, o adaptar los controles disponibles en la versión instalada. Registrar la configuración real con captura. sonar.qualitygate.wait=true hace fallar el comando si el gate no se aprueba. Si no es posible aprobarlo, adjuntar plan técnicamente justificado con responsable y fecha; no cambiar la evidencia a “aprobado”.

4.2 Hallazgos de dependencias: medición distinta

npm audit	Inicial	Después
Paquetes señalados	19	13
Críticos	1	0
Altos	11	7
Moderados	7	6
Dependencias de producción con hallazgos	No presentado como Sonar	0 en npm audit --omit=dev

Se actualizaron handlebars, flatted, lodash y node-forge mediante overrides y se volvió a ejecutar Newman. Persisten hallazgos transitivos de herramientas de desarrollo; el reporte final los enumera. npm audit es SCA y no reemplaza E12. La ausencia de hallazgos en dependencias de producción no demuestra ausencia de vulnerabilidades en el código.

5. E13 · Pipeline

Archivo: `.github/workflows/quality.yml`. Ejecutor requerido: runner autohospedado Windows X64 en la computadora de Erick con Git, Node 24 y Docker Desktop. Esto permite acceder al SonarQube local sin exponerlo a internet. El workflow está preparado y revisado, pero no ejecutado en GitHub.

Orden	Paso	Condición de avance
1	Compilación backend/frontend	Comandos terminan con código 0.
2	Vitest, cobertura y delta de línea base	90 pruebas y umbrales aprobados.
3	SonarQube y Quality Gate	Escaneo finaliza y gate no falla.
4	Newman + regresión HTTP	Colección y casos funcionales aprobados.
5	Despliegue de backend y frontend	Solo push; Render confirma live para el mismo SHA.

Secretos: `SONAR_TOKEN` y `RENDER_API_KEY`. Variables por ambiente: `SONAR_PROJECT_KEY`, `RENDER_BACKEND_ID` y `RENDER_FRONTEND_ID`. Sonar Community puede manejar proyectos separados para DEV/QA/PROD sin asumir soporte de análisis multirrama. En cada proyecto debe configurarse el gate. El job quality debe ser obligatorio en reglas de protección de QA/main; el YAML solo no instala esas reglas.

5.1 Cloud con persistencia

La plantilla `render.yaml` define un API Node y un frontend estático para QA. `DATABASE_URL` debe apuntar a PostgreSQL persistente con TLS, por ejemplo en Neon. En el servicio gratuito de Render el sistema de archivos es efímero, de modo que PGlite local no se usa como almacenamiento cloud. Los servicios gratuitos pueden suspenderse por inactividad: medir calentamiento y carga en condiciones declaradas. Consultar límites vigentes antes de crear los ambientes.

6. E14 · Carga

Elemento	Configuración
Herramienta	k6, script qa/carga/reservalab.k6.js
Objetivo	URL HTTPS real del API QA desplegado
Endpoints	GET /api/equipment y GET /api/dashboard
Perfil	0-50 VU en 1 min; 50 VU sostenidos 5 min; descenso 30 s
Duración mínima programada	6 min 30 s, más setup/graceful shutdown
Autenticación	Una cuenta de prueba en setup; token válido de ocho horas
Métricas	Promedio, p95, throughput (solicitudes/segundo), tasa de error
Umbrales RNF-01	p95<2000 ms por endpoint y tasa de errores<1%
Evidencia prevista	qa/evidencias/carga-k6.json
Resultado actual	No ejecutado: no existe aún ambiente cloud accesible.

```
k6 run qa/carga/reservalab.k6.js
```

Antes de ejecutar, definir BASE_URL, EMAIL y PASSWORD en el entorno de la terminal. El script usa HTTPS por defecto y no contiene contraseñas. No usar ALLOW_LOCAL para acreditar E14: esa opción solo permite una comprobación técnica local, que debe declararse como tal.

6.1 Conclusión que debe completarse con la medición

Registrar fecha, región, tipo de instancia, versión del producto, calentamiento, promedio/p95 por endpoint, solicitudes totales, duración efectiva y tasa de error. Comparar cada p95 con 2000 ms y la tasa con 1%. Si falla un criterio, concluir incumplimiento de RNF-01 para ese escenario y proponer optimización o ajuste de recursos. No se incluyen números simulados ni una conclusión de cumplimiento sin ejecución.

7. E15-E16 · Defensa y bitácora

El PDF 04 prepara una demostración de máximo diez minutos. El estudiante debe mostrar pipeline completo en vivo, resultados de Sonar y carga, con rostro visible durante su segmento. El PDF 06 es una ficha de media página para reflexión personal, no una bitácora escrita como si el estudiante ya hubiera realizado las actividades.

8. Estado por entregable

Entregable	Estado verificable
E9	Estrategia y workflow preparados; faltan ramas/remoto, protecciones y tres revisiones externas.
E10	90 unitarias y cobertura local verificadas; confirmar alcance crítico y ejecutarlas en pipeline.
E11	24 solicitudes y 85 aserciones verificadas localmente; falta corrida en pipeline.
E12	Configuración preparada; medición Sonar y gate pendientes.
E13	Pipeline escrito; ejecución y despliegues remotos pendientes.
E14	Script preparado; carga cloud y conclusión pendientes.
E15	Pauta preparada; video pendiente.
E16	Ficha preparada; reflexión personal pendiente.

9. Fuentes técnicas

SonarSource. JavaScript/TypeScript test coverage: <https://docs.sonarsource.com/sonarqube-cloud/analyzing-source-code/test-coverage/javascript-typescript-test-coverage>

SonarSource. CI integration:

<https://docs.sonarsource.com/sonarqube-community-build/analyzing-source-code/ci-integration/overview>

Postman. Newman: <https://learning.postman.com/docs/reference/newman-cli/installing-running-newman/>

Grafana. Umbrales k6: <https://grafana.com/docs/k6/latest/using-k6/thresholds/>

Render. Free services: <https://render.com/docs/free>

Render. Deployments: <https://render.com/docs/deploys>

Render. Blueprint: <https://render.com/docs/blueprint-spec>

Neon. PostgreSQL y plan gratuito: <https://neon.com/docs/introduction/plans>

Vitest. Coverage: <https://vitest.dev/config/coverage>

PGlite. API: <https://pglite.dev/docs/api>

Consulta de fuentes técnicas: 25 de septiembre de 2026. Los límites de proveedores y las versiones pueden cambiar. Las decisiones de arquitectura son del proyecto; no constituyen una certificación de los proveedores.